



← Back to student guides

[📈] MLflow

Tracking & registry

3 levels

111 sections



The Complete Guide

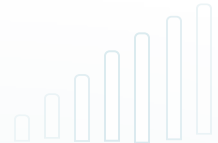
Taught at three levels, written three ways. Pick Beginner, Mid-level, or Senior, then read the full explanation, a fast interview review, or the practical tips and traps for that level.

RUN

MODEL

REGISTRY

SERVE



Quick Start

A one-page starter for this tool. Scroll through it here, or open it full screen.

Open PDF

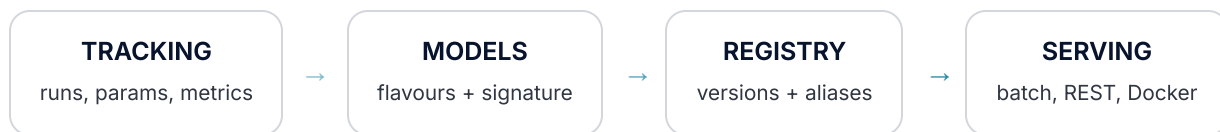






Part three of three, and the one to read if you only read one. A cumulative review of **the entire series** (foundations, packaging and promotion machinery, and the governance and scale work a senior owns) organised by topic rather than by level. About fifty minutes. Fast review first, common questions at the end.

Part one: foundations



MLflow has four components: Tracking for runs, Models for framework-agnostic packaging, a Registry for named versions and moveable aliases, and Projects for reproducible entry points. The core idea is the model format (a directory with an `MLmodel` manifest declaring flavours, a signature, and an environment) which is why one artifact loads in a batch job, a REST server, or a container.

Two stores: backend for metadata, artifact store for files. **The registry needs a database backend. Params immutable, metrics stepped, tags mutable. Consume by alias. MLflow records the commit, not a diff.**

The four things you log

Kind	Mutable	For
Param	No	Configuration: hyperparameters, seed, data partition
Metric	Yes, stepped	Anything measured
Tag	Yes	Organisation and search
Artifact	Append-only	Files, including the model



MLmodel

Copy

```
flavors:
  python_function: {loader_module: mlflow.sklearn, model_path: model.pkl}
  sklearn: {pickled_model: model.pkl, sklearn_version: 1.5.1}
signature: {inputs: '...', outputs: '...'}
run_id: a1b2c3d4e5f64718
```

URI	Points at
<code>runs:/<id>/model</code>	A model logged by a run
<code>models:/<name>/<n></code>	A pinned registry version
<code>models:/<name>@champion</code>	Whatever holds that alias

Native flavour returns the real object; **pyfunc** returns a generic `predict`, which is what every deployment target uses. **A signature is a contract**; without it, misordered or mistyped input scores silently.

The rules that never change

Context manager always. Named experiment, named runs, tags in code. Signature on every model. `mlflow.evaluate` for the standard metric set. Aliases, not deprecated stages. Commit before a run you might defend.

The traps

Symptom	Cause	Fix
No runs in the UI	Wrote to local <code>./mlruns</code>	Set <code>MLFLOW_TRACKING_URI</code>
<code>register_model</code> fails	File-based backend	Postgres/MySQL
Run stuck in <code>RUNNING</code>	No context manager	<code>with mlflow.start_run()</code>



Param already logged	log_param twice	Params once, metrics vary
Nothing autologged	<code>autoLog()</code> after the estimator	Call it beside the imports
Serving fails to load	Environment mismatch	Default <code>virtualenv</code> , not <code>local</code>

Part two: packaging and promotion machinery

Runs, nesting, and the client API

Nesting is the `mlflow.parentRunId` tag, so children are queryable. Use the **client API** for threads, other runs, bulk `log_batch`, and all registry work, because the fluent API is global state keyed on the active run.

Per-batch metric logging is why teams think MLflow is slow

One `log_metric` per batch is one HTTP round trip per point. Log per epoch, or accumulate and use `log_batch`. This is the most common scale mistake and a frequent interview probe.

Autologging internals

Patches framework functions at import time. Three behaviours: **it creates a run if none is active; it must be enabled before the estimator exists; `log_input_examples=True` embeds real training data** in the artifact. Set `log_models=False` and log the model yourself, or you get two model directories with no statement of which is authoritative.

Custom pyfunc



```
class ChurnModel(mlflow.pyfunc.PythonModel):
    def load_context(self, context): ...
    def predict(self, context, model_input, params=None): ...

mlflow.pyfunc.log_model(name="model", python_model=ChurnModel(),
    artifacts={"scaler": ..., "classifier": ...},
    code_paths=["src/features.py"], signature=signature,
    pip_requirements="requirements-serving.txt",
    registered_model_name="churn-classifier")
```

Piece	Role
<code>load_context</code>	One-time setup from declared artifacts
<code>predict(ctx, input, params)</code>	The whole inference contract, including postprocessing
<code>code_paths</code>	Your modules, so the model imports nothing from your repo
Signature <code>params</code>	Validated call-time knobs: threshold, top-k, temperature

Missing `code_paths` is the top cause of "loads locally, fails in serving", because the class is pickled by reference, so the module must be importable.

Dependencies

Inferred by introspecting loaded modules. **Inference sees imports, not intent**, so dynamic or conditional imports are invisible. Controls: `extra_pip_requirements`, an explicit `pip_requirements`, `get_model_dependencies`, `validate_serving_input`, and `mlflow models predict --env-manager virtualenv` to reproduce the serving path.

Gates

PYTHON

Copy

```
mlflow.evaluate(model=candidate, data=holdout, targets="label", model_type="classifier",
    baseline_model=champion,
```



Raises `ModelValidationFailedException` , so CI fails with the metric named. **Relative to a baseline** is the honest gate; an absolute floor stops discriminating once cleared. An aggregate metric hides a broken segment.

The registry as a workflow



1 Register

Training registers a version tagged `validated=false` .



2 Validate

Thresholds on a frozen holdout; tag the result.



3 Compare

Against the champion with a minimum relative change.



4 Promote

`previous` keeps the old version; `champion` moves.



5 Roll back

Move `champion` to `previous` . One call, no redeploy.

Datasets, deployment, tracing

`mlflow.data.from_pandas(...)` plus `log_input` records name, source, **digest**, schema, and profile. It is **lineage annotation, not versioning**. Nothing stops the source URI being overwritten.

Targets: batch `pyfunc.load_model` , `mlflow models serve --workers N --enable-mlserver` , `build-docker` , `spark_udf` , and the deployments API. `/invocations` takes `dataframe_split` , `dataframe_records` , `instances` , `inputs` , plus `params` .



name the judge.

A signature change is a breaking API change that skips code review

Promotion moves an alias with no deploy step, so a renamed column reaches production without an API diff being reviewed. Additive-only, or a new registered model name.

Part three: trust, scale, and ownership

The trust model

Moving an alias changes production, and by default nothing stops anyone with tracking credentials from doing it.

Actor	Needs	Must not have
Researcher	Create runs, log models, register versions	Move <code>champion</code> ; delete
CI on a pull request	Create runs in <code>*/ci</code>	Register or alias
CI on a protected branch	Register and alias, from a gated job	Delete
Serving	Read registry and artifacts	Any write
Retention	Scoped, audited delete	Register or alias

Three sharpening properties: **authorisation is coarse** (per-experiment and per-model permissions, not rich RBAC, and unauthenticated by default); **clients touch object storage directly**, so everyone needs bucket credentials unless you proxy; and **a pickle is executable**, so loading a model runs code.

Auth and proxied artifacts



```
mlflow server --backend-store-uri postgresql://... \  
--artifacts-destination s3://ml-artifacts/mlflow --serve-artifacts \  
--app-name basic-auth --workers 4
```

Permission	Allows
READ	View runs, metrics, models
EDIT	Create and modify runs and versions
MANAGE	Change permissions, delete

`--serve-artifacts` means only the server holds bucket credentials and artifact access follows MLflow's permissions, at the cost of making the server a throughput bottleneck and single point of failure for transfers.

Self-hosting

Component	If it dies	Grows with
Reverse proxy	No access; stores intact	-
Tracking server (stateless)	Restart or scale out	-
Backend database	Everything stops: system of record	Metric points × runs
Artifact store	Models unreadable; metadata survives	Artifacts × retention

SQLite in a shared deployment is a time bomb: `database is locked` under concurrent writers. Postgres from the second user onwards. Pin the server image tag: `latest` means an unplanned schema migration on the next restart.

Backups and upgrades



missing models. verify. login → a run's metrics → alias resolution → a model download → a new run logging. Upgrades run migrations: back up, `mlflow db upgrade` deliberately, server before clients.

Multi-tenancy

Open-source MLflow is one server with coarse permissions. Isolation comes from experiment naming conventions, the basic-auth plugin, artifact prefixes with bucket policy, mandatory tags, and, for hard isolation, a separate server and database per tenant. A thirty-line wrapper turns every convention into a default.

Cost and retention

Centre	Driver	Control
Artifact storage	Models per run × runs × retention	Log models deliberately, lifecycle, <code>mlflow gc</code>
Backend database	Metric points × runs	Per-epoch logging, retention on CI and sweeps
Server	Concurrency, proxied throughput	Workers, replicas

Two retention traps

`delete_run` is a **soft** delete. Nothing is reclaimed until `mlflow gc` runs. A lifecycle rule can expire the object behind a registered version, leaving an entry that resolves and then fails to download, discovered by a batch job at 3am.

Scale, in order of what breaks

1 Metric write rate



2 Run count

Slow search and table. Fix: retention, then `mlflow gc`.

3 Artifact volume

Cost and proxied throughput. Fix: lifecycle, fewer logged models.

4 Client concurrency

Worker saturation. Fix: more workers and replicas. It is stateless.

The backend database is the single point of failure, and it includes alias resolution, so serving should resolve the alias once at startup, not per request.

Reproducibility

Layer	Recorded	Pin with
Code	The commit only , no diff	A dirty-tree guard
Packages	Inferred pins	An explicit committed serving requirements file
CUDA, system libs	No	The base image by digest
Data	Only via <code>log_input</code>	A dated immutable source, digest, and a param
Seeds	Only if logged	Log and set them

MLflow will record an unreproducible run with no warning. The checks are yours: refuse to register from a dirty tree, require a dataset input, and run a **weekly rebuild check** against the champion with a tolerance.

Governed promotion, lineage, erasure

Three mechanisms make promotion auditable: **permissions** (only a CI account may move aliases), a **gated workflow** (protected branch with required review), and a `promotion_audit`



Audit chain: model → version → run → code, data, configuration. A lineage script proves whether each link exists. Common gaps: a dirty tree, no dataset recorded, no approval trail.

Erasure exposure specific to MLflow: **prediction artifacts** with identifiers, `input_example` **embedded in a model**, and **GenAI traces** holding prompts verbatim. Weights are un-erasable in practice, and retraining is the remedy.

Serving and the supply chain

Authenticate at an ingress; the scoring server authenticates nobody. Resource limits, pinned base image by digest, resolve the alias at startup, and a rehearsed rollback.

A registry version is immutable metadata pointing at mutable bytes

MLflow will not let you change version 7, but anyone with bucket write access can replace the object it points at, and every consumer then loads different weights with no error. Write-restrict the prefix, enable versioning, prefer proxied access.

Where MLflow stops

Need	Better tool
Executing and scheduling	Airflow, Dagster, Prefect, Argo, CI
Data versioning with immutability	DVC, LakeFS, Delta/Iceberg
Point-in-time features	Feast, Tecton
High-scale low-latency serving	KServe, Seldon, Triton, managed endpoints
Fine-grained RBAC	Managed MLflow, or a server per tenant
Hyperparameter search	Optuna, Ray Tune: then log to MLflow



01 Design MLflow for a team of thirty across three squads.

Self-hosted behind TLS and auth, Postgres backend with disk and connection alerts, artifacts in object storage with a prefix per team, bucket versioning on, and proxied artifact access so clients hold no bucket keys. Experiments named `team/project/purpose`, mandatory `team` and `cost_centre` tags, and a shared wrapper making all of that default. Researchers get `EDIT` on experiments and `READ` on registered models; only a CI service account can move aliases, from a reviewed workflow that writes an audit tag. Scheduled retention plus `mlflow gc`, a weekly rebuild check, and a published platform report. Then say plainly that authorisation is coarse, so isolation is conventions, permissions, and buckets, not roles.

02 Why is alias permission the security question that matters?

Because consumers load `models:/name@champion`, so moving that alias changes what production serves with no deploy and no code review. On a default install anyone with tracking credentials can do it, and nothing records who did. That makes alias write the production-change surface, and it should belong to one gated CI job rather than to every researcher.

03 What is MLflow's most surprising architectural detail?

By default the tracking server returns an artifact URI and the client reads and writes object storage itself. So every user and every CI job needs bucket credentials, and artifact access is governed by bucket policy rather than by MLflow permissions. It is why "metrics appear but artifacts 404" is the most common support question, and `--serve-artifacts` with `--artifacts-destination` is the fix, at the cost of routing all transfers through the server.

04 Which component breaks first as you grow, and why?

The backend database, on metric write rate. Per-batch logging from several concurrent runs produces enormous write volume, which slows logging and then the UI; after that it is run count making search slow. Both are fixed by client behaviour first, per-epoch logging and retention, before adding database resources. Because the database also serves alias resolution, a database problem is a production problem unless serving caches its model at startup.



plugin, plus the artifact bucket with versioning and ideally replication. Metadata alone restores a complete-looking system where every model resolves and nothing downloads. That is why "can you download a known model" belongs in the restore verification, alongside login, a run's metrics, and alias resolution.

06 Why is SQLite unacceptable for a shared server?

It locks under concurrent writers, so several people training at once produce `database is locked` exactly when the server is most needed. It also cannot be replicated or scaled, and migrating a live SQLite store to Postgres involves downtime and care. Postgres from the second user onwards.

07 How do you make a model from a year ago rebuildable?

MLflow records the commit, params, and pip pins; everything else is yours. Refuse to register from a dirty tree, require a logged dataset with a dated immutable source and a digest, pin the base image by digest, log and set the seed, and keep an explicit committed serving requirements file. Then schedule a rebuild check that reruns the champion's recipe and compares the metric within a tolerance, because unpinned layers degrade silently and the first run of that check will fail.

08 Why does MLflow's lack of a stored diff matter?

Because a run tagged with a commit but executed from a dirty tree looks as trustworthy as a clean one, and nothing warns you. Unlike tools that capture the working tree, MLflow cannot reconstruct what ran. The mitigation is procedural: a guard that refuses to create a promotable run from a dirty tree, and a `git_clean` tag that promotion checks.

09 How do you build an approval trail for promotion?

Three things together. Permissions, so only a CI service account can move an alias. A gated workflow on a protected branch, so the human approval is a reviewed pull request by someone other than the author. An audit tag on the model version recording who, when, which frozen holdout, which baseline version, which thresholds, and the resulting metrics. The tag is the record; the permission is the control, because a tag written by the person who trained the model is a self-approval.



One generated report: the alias, the version, the version tags, the producing run with its user and status, the git commit and whether the tree was clean, the dataset inputs with their sources and digests, the params as executed, and the promotion audit tag. If any of those is missing, usually the data chain or the approval, that is the finding, and it is better to know it before the audit than during it.

11 How would you handle an erasure request?

Enumerate the MLflow-specific copies first: prediction artifacts containing identifiers, an `input_example` embedded in a model if autologging left it on, GenAI traces holding prompts verbatim, and database backups. Then be honest that weights retain the influence and are sometimes extractable, so retraining is the only real remedy for the model itself. The architectural fix is upstream: keep identifiers out of logged artifacts, turn off input examples, and decide trace redaction before enabling tracing on real traffic.

12 What is the supply-chain risk in an MLflow model?

Loading a model executes code: unpickling is code execution, and `code_paths` ships arbitrary modules that run at load time. Worse, a registry version is immutable metadata pointing at mutable bytes: anyone with bucket write access can replace the object behind version 7 and every consumer loads different weights with no error. Write-restrict the artifact prefix, enable versioning, prefer proxied access, and treat an external model artifact like an unreviewed dependency.

13 Design a promotion gate a reviewer would trust.

Evaluate on a frozen holdout the model has never seen, with the current champion as `baseline_model` and a `min_relative_change` so noise does not pass. Include a business-cost metric and a worst-segment metric with their own thresholds, because an aggregate improvement can hide a regression for one group. Fail on the exception rather than on a logged number, and write the audit tag. Then rehearse the rollback.

14 How do you roll back, and how fast?

Move `champion` to the version held by `previous`. That is one call, and it only works because `previous` was set at promotion time. If serving caches its model at startup, the rollback also needs a restart or a cache expiry, which is a trade-off worth making explicit,



15

What is your retention policy, and what could it break?

Runs in CI experiments deleted after thirty days, research runs after a year, with exemptions for tagged runs and anything a registered version references, then `mlflow gc` to reclaim space, because `delete_run` is only a soft delete. On the artifact side, lifecycle rules per prefix with the registry prefix exempt from expiry, because expiring the object behind a registered version produces a resolvable entry that fails to download.

16

How do you keep researchers productive without giving everyone everything?

A wrapper: they call one function and get the right experiment path, mandatory tags, registered model name, serving requirements, and image digest. They can create runs and register versions; they cannot move an alias. Promotion is a reviewed workflow. The result is that the safe path is also the shortest path, which is the only governance that survives a deadline.

17

What monitoring does an MLflow platform need?

Backend database disk above 80% first, because when it fills, logging, the UI, and alias resolution all stop. Then database connections, tracking-server 5xx rate, artifact-store error rate, the rebuild-check result, storage growth per team, and a notification on any production alias change. Deliberately not on that list: users' failed runs, which belong to their authors and will drown the real alerts.

18

You inherit an MLflow setup. What do you check first?

Whether a researcher can move a production alias, and whether a restore has ever been performed. Those two are "anyone can change production" and "a bad day is permanent". Then: unauthenticated server, SQLite backend, artifact root on a server disk, unpinned server image, no bucket versioning, no retention, and whether promotion is a script with a threshold or a person clicking.

19

Where does MLflow stop?

It executes nothing (no agent, no queue, no retries) so orchestration is Airflow, Dagster, or CI. Its dataset tracking is annotation, not versioning, so immutability comes from DVC, LakeFS, or a table format. It is not a feature store and not a serving mesh. Authorisation is



20

What is the single idea that connects the whole series?

A model is only useful if something other than its author can load it, and only trustworthy if something other than its author's memory explains it. Signatures, flavours, pinned environments, aliases, audit tags, backups, and rebuild checks all exist to keep that true as the number of models, people, and months grows.

Final self-test

- Give the thirty-second answer, the four components, and the two stores.
- State the rules that never change: context manager, signature, alias, evaluate, commit.
- Explain what `log_model` writes and why pyfunc makes it portable.
- Say why per-batch metric logging is the classic scale mistake.
- Explain custom pyfunc, `code_paths`, and signature `params`.
- Describe how requirements are inferred and the one case that always fails.
- Give the command that reproduces the serving environment without a server.
- Explain `baseline_model` plus `min_relative_change`, and why an aggregate metric is not enough.
- Say why alias write is the production-change surface, and how you restrict it.
- Name MLflow's most surprising architectural detail and the error it causes.
- Say which component breaks first at scale and the fix order.
- List the two backup parts and the restore step that usually fails.
- Name every layer that must be pinned for an eighteen-month rebuild, and how you verify it.
- Explain why a registry version is immutable metadata over mutable bytes.
- Name three places MLflow is the wrong tool and what you would use instead.



MLOps MENA
COMMUNITY

العربية



[Join the Community ↗](#)



Senior · Detailed

Senior · Tips & Tricks



MLOps MENA
COMMUNITY

Free MLOps and AI learning for engineers across the Middle East and North Africa — live sessions, three structured roadmaps, open cohort courses, study groups, and mentorship. We also train, staff, and build for companies across the region.



LEARN

[Roadmaps](#)

[Student Guides](#)

[Courses](#)

[The MLOps Practitioner](#)

[Sessions](#)

[Articles](#)

COMMUNITY

[Meet the team](#)

[Services for companies](#)

[FAQ](#)

[Contact us](#)

[Privacy Policy](#)



MLOps MENA
COMMUNITY

العربية



Join the Community ↗



Discord

LinkedIn

X

YouTube

GitHub

Zomra



Brainsmingle space

Free to join this week with code **FREECORETEAM**

© 2026 MLOps MENA Community. Free learning for the community.

Built by the community, for the community.